

نظام إدارة المهام المتكامل

TASK MANAGEMENT SYSTEM

دليل البنية البرمجية المتكامل والتوثيق التقني التفصيلي للمشروع

تحديث وتجميل تنسيق صفحات الأقسام والبطاقات

إعادة هيكلة طبقة الخدمات لإكمال المهام الفرعية تلقائياً

دمج العداد التنازلي التفاعلي ونظام التحذيرات اللحظي

تطوير وإعداد:

كريم رمضان والمساعد البرمجي الذكي AntigraVity

التقنيات المستخدمة:

Laravel 13 | Alpine.js | CSS Glassmorphism | SQLite

جدول المحتويات

3. ص.	1. الخلاصة التنفيذية ورؤية المشروع وأهدافه
3. ص.	2. البنية البرمجية ونمط التصميم (MVC + طبقة الخدمات)
4. ص.	3. هيكل قاعدة البيانات، العلاقات وال Migrations
5. ص.	4. شرح تفصيلي لملفات وقواعد كود المشروع ملغا بملف
8. ص.	5. تفاصيل تطبيق الميزات الاحترافية المضافة حديثا للمشروع
8. ص.	5.1. العداد التنازلي التفاعلي اللحظي ونظام التحذير عند اقتراب المهلة
9. ص.	5.2. خطاف الحذف وإكمال المهام الفرعية تلقائيا في طبقة الخدمات
9. ص.	5.3. حساب تقدم المهام الإجمالي على مستوى النظام والبطاقات
9. ص.	5.4. تصميم بطاقات الأقسام والتنسيق الجمالي العام للمشروع
10. ص.	6. توصيات استراتيجية للأداء العالي والنمو المستقبلي للمشروع

1. الخلاصة التنفيذية ورؤية المشروع وأهدافه

المنظمات التقليدية التي تبدو جامدة ومملة، تم تصميم هذا التطبيق ليتفاعل مع نشاط المستخدم بشكل لحظي وحيوي. الإنتاجية الفردية والجماعية، مع التركيز التام على تقديم تجربة مستخدم استثنائية (Premium User Experience). بخلاف نظام إدارة المهام هو تطبيق ويب متطور وعالي الأداء تم تصميمه لتنظيم وجدولة المهام وتحسين ملمس تطبيقات الصفحة الواحدة (SPA) بحيث تتم كل التحديثات والعمليات فوراً دون الحاجة لإعادة تحميل الصفحة بالكامل. إطار العمل 13 Laravel في الواجهة الخلفية ويوظف تقنيات Javascript حديثة و Axios في الواجهة الأمامية، مما يمنح التطبيق يعتمد النظام بالكامل على

الركائز التقنية الأساسية للمشروع:

- المتينة: فصل منطق العمل تماماً عن المتحكمات (Controllers) وتوجيهه داخل طبقة خدمات مخصصة (TaskService).
- البنية البرمجية
- للحظية (Real-time): تحديث وحفظ البيانات وجلب الإحصائيات مباشرة وبسرعة فائقة عبر طلبات Axios غير المتزامنة.
- التفاعلية
- الجمالية والجمال البصري الفاخر: اعتماد لوحة ألوان هادئة ونظام بطاقات تفاعلية مع حواف متوهجة وتأثيرات انتقالية ناعمة.
- المباشر: إضافة عداد تنازلي حي يعمل في خلفية المتصفح ويتحول تلقائياً إلى وضع الطوارئ عند اقتراب انتهاء المهلة.
- نظام الإنذار والمؤقتات

2. البنية البرمجية ونمط التصميم (MVC + طبقة الخدمات)

(Eloquent Models) مما ينتهك مبدأ المسؤولية الفردية، يعتمد التطبيق نمط طبقة الخدمات (Service Layer Pattern). منطق الأعمال مباشرة داخل المتحكمات (Controllers) مما يجعل الكود ضخماً وصعب الصيانة، أو حشره داخل الموديلات يلتزم المشروع التزاماً صارماً بأفضل المعايير الهندسية لتطوير البرمجيات. بدلاً من تكديس

Unified Data & Process Flow Model

[Client UI]	<--->	[TaskController]	<--->	[TaskService]	<--->	[Eloquent Models]
(Blade & JS)		(Handles Requests)		(Business Logic)		(Database Storage)

دوره فقط في استقبال طلبات HTTP والتحقق من صحتها وتفويض العمل لـ TaskService ثم إرجاع استجابة API موحدة. والمسؤول الوحيد عن عمليات إنشاء وتعديل وحذف المهام وإطلاق العمليات الجانبية. أما المتحكم (TaskController) فينحصر كلاس TaskService هو المايسترو

3. هيكل قاعدة البيانات، العلاقات والمigrations

العلائقية (Relational Database) مثل SQLite أو MySQL. يتكون النظام من أربعة كيانات أساسية مرتبطة برمجيا كالتالي:
تم تصميم قاعدة البيانات بهيكل علاقات قوي ومحسن ليتناسب مع قواعد البيانات

- الكاملة للمهام ويتم فحص هويته برمجيا في كل طلب. (One-to-Many) المستخدم: المستخدم يملك مهام متعددة (User - يملك الصلاحية
- اختياري. كما أنها تملك مهام فرعية متعددة (Category) المهمة: المهمة تنتمي لمستخدم معين وقسم (Task - (One-to-Many).
- المهمة (is_completed) المهمة الفرعية: المهمة الفرعية تنتمي لمهمة رئيسية، وتحمل حالة اكتمال بولينية (Subtask - تعبر عن جزء من إنجاز
- القسم: القسم ينتمي لمستخدم معين، ويستخدم لتصنيف وتنظيم المهام داخل لوحة التحكم (Category -

أكواد الميجريشنز لجدول المهام والمهام الفرعية:

Eloquent Migrations (Tasks & Subtasks Tables)

```
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->text('description')->nullable();
    $table->timestamp('due_date');
    $table->enum('priority', ['low', 'medium', 'high'])->default('medium');
    $table->enum('status', ['pending', 'in progress', 'completed'])->default('pending');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->foreignId('category_id')->nullable()->constrained()->nullOnDelete();
    $table->timestamps();
});

Schema::create('subtasks', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->boolean('is_completed')->default(false);
    $table->foreignId('task_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

4. شرح تفصيلي لملفات وقواعد كود المشروع ملغا بملف

فيما يلي جرد تفصيلي وتحليل لجميع الملفات والوحدات البرمجية الأساسية في المشروع مع توضيح وظيفة كل منها:

1. app/Services/TaskService.php

هو الملف المركزي لكافة العمليات البرمجية والتفاعلية مع قاعدة البيانات. يحتوي على الدوال التالية:

- `getAllTasks()`: لجلب وتصفية مهام المستخدم الحالي مع دعم البحث بالكلمات والفلتر بالاقسام والحالة.
- `createTask()`: لإنشاء مهمة جديدة في قاعدة البيانات وإضافة مهامها الفرعية المرفقة بشكل متزامن وآمن.
- `updateTask()`: لتحديث بيانات المهمة وإدارتها وإعادة معالجة المهام الفرعية داخل معاملة (Database Transaction).
- `deleteTask()`: لحذف المهمة ونقلها إلى سلة الحذف الناعم (Soft Delete) عبر استدعاء `Task::destroy()` لحل مشاكل ال IDE.

2. app/Http/Controllers/TaskController.php

المتحكم الرئيسي للمهام والواجهات. يستخدم `Trait ApiResponse` لتنسيق استجابات الويب وال API بشكل موحد:

- `index()`: جلب وعرض لوحة التحكم الرئيسية وإحصائيات.
- `store()`, `update()`, `destroy()`: لمعالجة وتطبيق الإدخالات مع قواعد تحقق صارمة ثم استدعاء طبقة الخدمات.
- `today()`, `upcoming()`, `important()`, `completed()`: واجهات مخصصة لعرض القوائم المصنفة مع تمرير متغيرات التظليل النشط لل Navigation.

3. app/Http/Controllers/CategoryController.php & SubtaskController.php

- `CategoryController`: يتحكم في إنشاء الأقسام وعرضها وحذفها لإبقاء لوحة التحكم منظمة وجميلة.
- `SubtaskController`: التقدم المحدثة فوراً للواجهة (POST) يتحكم في استقبال إشارات تبديل حالة إنجاز المهام الفرعية وإرجاع نسبة.

4. routes/web.php

ونقاط الوصول في النظام مع حماية المسارات الهامة وإخضاعها لجدار حماية المصادقة (Auth Middleware) الخاص بـ Laravel. يحدد كافة مسارات

Core Task and Subtask Endpoints

```
Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('/Dashboard', [TaskController::class, 'index'])->name('dashboard');
    Route::get('/Today', [TaskController::class, 'today'])->name('today');
    Route::get('/Upcoming', [TaskController::class, 'upcoming'])->name('upcoming');
    Route::get('/Important', [TaskController::class, 'important'])->name('important');
    Route::get('/Completed', [TaskController::class, 'completed'])->name('completed');

    Route::post('/tasks', [TaskController::class, 'store'])->name('tasks.store');
    Route::put('/tasks/{id}', [TaskController::class, 'update'])->name('tasks.update');
    Route::delete('/tasks/{id}', [TaskController::class, 'destroy'])->name('tasks.destroy');
    Route::post('/subtasks/{id}/toggle', [SubtaskController::class, 'toggle'])->name('subtasks.toggle');
});
```

5. resources/views/Dashboard.blade.php

بطاقة المهمة بالكامل وتحديثها فوراً في الصفحة عبر خاصية taskEl.outerHTML دون الحاجة إطلاقاً لإعادة تحميل الصفحة. أو إكمالها، يتواصل Axios بلطف مع الـ Controller. وعند استقبال الرد، تقوم دالة buildTaskHtml(task) في الـ JS بإعادة بناء بين واجهات Blade الأنيقة وبين تقنيات Alpine.js و Axios. عندما يقوم المستخدم بإجراء عمليات إضافة مهمة أو تعديلها ملف العرض الأساسي والتفاعلي للمشروع. يجمع

6. resources/views/Categories.blade.php

تم تعديل لون خلفية بطاقات التصنيفات في هذا الملف إلى اللون الفاخر (#ecec) لتوفير تباين بصري ممتاز ومريح للغة. يعرض قائمة تصنيف الأقسام التي ينشئها المستخدم.

7. public/css/style.css

الرئيسي للمشروع. يحتوي على تعريف المتغيرات والأنيميشن والتأثيرات الزجاجة وتخطيطات الشاشات المتجاوبة بالكامل. ملف التنسيقات

5. تفاصيل تطبيق الميزات الاحترافية المضافة حديثا للمشروع

قمنا بتنفيذ أربع ميزات ابتكارية متطورة في الكود البرمجي للمشروع. فيما يلي التحليل التقني لكل ميزة وكود عملها: لترقية تجربة الاستخدام إلى المستوى الاحترافي،

5.1. العداد التنازلي التفاعلي اللحظي ونظام التحذير عند اقتراب المهلة

data-status و data-due. ثم يقوم كود ال Javascript بتشغيل حلقة عداد تنازلي في خلفية المتصفح كل ثانية كالتالي: تمت إضافة شريط عداد تنازلي لكل بطاقة مهمة. يقوم الخادم بطباعة وسم العداد محملا بسمات البيانات

JavaScript Countdown Loop & Icon Animation Handler

```
function updateCountdowns() {
  document.querySelectorAll('.task-countdown').forEach(el => {
    const dueDate = new Date(el.dataset.due);
    const diffMs = dueDate - new Date();

    if (diffMs <= 0) {
      // Task is Overdue
      el.closest('.task').classList.add('overdue');
      el.querySelector('.countdown-text').innerText = 'Overdue';
      el.querySelector('i').className = 'fa-solid fa-circle-exclamation';
      return;
    }

    // format days/hours/minutes/seconds...
    el.querySelector('.countdown-text').innerText = formatTime(diffMs);

    if (diffMs < 24 * 60 * 60 * 1000) { // Under 24 hours
      el.classList.add('urgent');
      el.querySelector('i').className = 'fa-solid fa-hourglass-start fa-spin';
    } else {
      el.classList.remove('urgent');
      el.querySelector('i').className = 'fa-solid fa-hourglass-half';
    }
  });
}

setInterval(updateCountdowns, 1000);
```

تلقائيا. وإذا انتهى الوقت، يتحول العداد فورا وتفاعليا إلى اللون الأحمر التحذيري الفاخر Overdue دون أي تحديث للصفحة. العداد في حالة الطوارئ (Urgent): تصبح الخلفية برتقالية دافئة، تبدأ البطاقة بالنبض التفاعلي، وتدور أيقونة الساعة الرملية إذا كان الوقت المتبقي أقل من 24 ساعة، يدخل

5.2. خطاف وإكمال المهام الفرعية تلقائيا في طبقة الخدمات (TaskService)

المهمة الكبيرة إلى مكتملة (Completed)، يتم تلقائيا إكمال كافة المهام الفرعية (Subtasks) التابعة لها في قاعدة البيانات: لسلسلة الاستخدام، قمنا ببرمجة خطاف برمجي ذكي في كلاس الخدمات بحيث بمجرد تحويل حالة

Backend Auto-Completion Subtask Hook

```
// Inside app/Services/TaskService.php@updateTask
public function updateTask(Task $task, array $data)
{
    DB::transaction(function () use ($task, $data) {
        $task->update($data);

        // Rule: If main task is completed, mark all subtasks completed!
        if (isset($data['status']) && $data['status'] === 'completed') {
            $task->subtasks()->update(['is_completed' => true]);
        }

        // rule: If subtasks are submitted in array, process them...
    });
    return $task->load('subtasks');
}
```

5.3. حساب تقدم المهام الإجمالي على مستوى النظام والبطاقات

قمنا بترقية تصميم العداد عند اكتمال المهمة ليتحول تلقائيا إلى شريط أكاديمي أخضر يحمل علامة صح تفاعلية Completed. المهام دون استثناء. للمهام التي لا تملك مهام فرعية، يتم حساب تقدمها كـ 100% في حال اكتمالها، و 0% للمعلقة. كما سابقا، كان يظهر شريط التقدم فقط للمهام التي تملك مهام فرعية. قمنا بإلغاء هذا الشرط ليظهر شريط التقدم لجميع بطاقات

5.4. تصميم بطاقات الأقسام والتنسيق الجمالي العام للمشروع

باللون الفاخر الهادئ (#ecec) مع إطار رمادي ناعم وشفاف يتوافق تماما مع الهوية البصرية الجمالية للوحة التحكم. تمت ترقية تصميم بطاقات الأقسام في واجهة Categories لتكتسي

6. توصيات استراتيجية للأداء العالي والنمو المستقبلي للمشروع

عمل التطبيق بكفاءة متناهية وسرعة خارقة (أقل من 50 مللي ثانية) مع زيادة أعداد المهام والمستخدمين، نوصي بالتالي: لضمان استمرار

البيانات: إضافة فهرس (Indexes) على الأعمدة الأكثر استخداما في الفترة مثل (user_id, status, priority, due_date). - تحسين استعلامات قاعدة

محرك Laravel Cache (مثل Redis) لتخزين أعداد المهام والعدادات النشطة للمستخدم وتحديثها فقط عند تغيير البيانات. - الذاكرة المؤقتة (Database Caching): استخدام

عند إضافة تنبيهات البريد الإلكتروني أو الرسائل النصية للمهام مستقبلا، يجب تفويضها لخلفية النظام لضمان سرعة التصفح. - نظام الطوابير (Laravel Queues):

(Vite build): تشغيل أمر 'npm run build' لضغط وتصغير ملفات الـ JS والـ CSS لضمان تحميل الصفحة في جزء من الثانية. - تهيئة أصول الواجهات

تم إنجاز التوثيق التقني بنجاح!

يمثل هذا الملف التوثيق البرمجي الكامل والنهائي لجميع الإضافات والأكواد التي تم بناؤها في التطبيق بنجاح.